UNIVERSIDAD TECNOLOGICA NACIONAL

Trabajo Práctico Integrador
Sistema de Gestión Hospitalaria

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Materia
Programación II

Docente
Torres, Matias Santiago

Alumno
Emperador, Javier

16 de noviembre de 2026

1. **Descripción del sistema**

El sistema modela el funcionamiento interno de un hospital en el lenguaje de programación Java.
Organiza médicos y pacientes a través de departamentos y salas, y coordina citas médicas entre ellos. Todo el procesamiento ocurre en memoria: los datos existen mientras el programa se ejecuta y no se persisten en archivos ni bases de datos.

Este mismo responde a las siguientes consultas del dominio:

- ¿En qué departamento trabaja este médico?
- ¿Qué pacientes tiene registrados el hospital?
- ¿Cuál es la historia clínica de este paciente?
- ¿Está disponible este médico o esta sala para una cita en determinada fecha y hora?

2. **Estructura del proyecto**

Los paquetes siguen la estructura obligatoria del enunciado (sección 2.4.)

```
src/main/java/org/jcr/
├── MainPruebas.java          (provisto por la cátedra, sin modificar)
├── entidades/                Persona, Paciente, Medico, Matricula, HistoriaClinica,
│                             Hospital, Departamento, Sala, Cita
│                             Validaciones (clase auxiliar interna al paquete)
├── enums/                    TipoSangre, EspecialidadMedica, EstadoCita
├── servicios/                CitaService, CitaManager
└── excepciones/              CitaException
```

Se agregó la clase auxiliar **Validaciones** dentro del paquete **entidades** para centralizar los chequeos repetidos (texto obligatorio, nulos, dígitos) y evitar su duplicación en cada contructor.

Validaciones no lleva modificador public: tiene visibilidad de paquete, lo que la hace accesible solo desde clases dentro de **org.jcr.entidades**. Es un utilitario interno del sistema, no parte del contrato público. Hacerla **public** permitiría que código externo la use, lo cual no tiene sentido para una clase que solo existe para no repetir lógica de validación en cada constructor.

3. **Enumeraciones**

Se implementaron las tres enumeraciones en el paquete org.jcr.enums:

Enumeración	Constantes	Descripción representativa
`TipoSangre`	8	`A_POSITIVO` → `"A+"`
`EspecialidadMedica`	12	`CARDIOLOGIA` → `"Cardiología"`
`EstadoCita`	5	`NO_ASISTIO` → `"No Asistió"`

Cada enumeración guarda su descripción en un atributo **private final String descripcion**, asignado por el constructor del enum y expuesto mediante **getDescripcion()**. Las descripciones incluyen tildes y signos exactos conforme al enunciado (sección 3.1).

4. **Herencia y encapsulamiento**

Jerarquía de clases

```
Persona (abstracta)
├── Paciente
└── Medico
```

Persona es abstracta: no puede instanciarse sola. Define los cinco datos comunes (nombre, apellido, DNI, fecha de nacimiento y tipo de sangre) con atributos **protected** y un constructor que los valida. **Paciente** y **Medico** extienden **Persona**, llaman a **super(...)** y agregan sus propios atributos sin redeclarar los heredados.

Métodos calculados

- getNombreCompleto(): concatena nombre, espacio y apellido. Devuelve "María López".
- getEdad(): calcula los años cumplidos a la fecha actual. Resta un año si el cumpleaños de este año todavía no llegó (se compara mes y día, no solo el año):

```
public int getEdad() {
    LocalDate hoy = LocalDate.now();
    int anos = hoy.getYear() - fechaNacimiento.getYear();
    if (hoy.isBefore(fechaNacimiento.plusYears(anos))) anos--;
    return anos;
}
```

Número de historia clínica

HistoriaClinica.getNumeroHistoria() devuelve **"HC-" + paciente.getDni() + "-" + año**. El año es el de creación de la historia (guardado en **fechaCreacion**), no el de nacimiento del paciente ni un valor fijo.

Encapsulamiento

Todos los atributos son **private** (o **protected** en **Persona**). No hay setters públicos que permitan saltear las validaciones. Las listas internas se exponen con **Collections.unmodifiableList()** o con una copia, nunca directamente.

5. Validaciones

Todas las validaciones se hacen dentro del constructor antes de asignar cualquier atributo. Si alguna falla, se lanza **IllegalArgumentException** y el objeto no llega a crearse.

La clase auxiliar **Validaciones** (visibilidad de paquete) centraliza las reglas repetidas:

Método	Qué valida
`textoObligatorio(valor, mensaje)`	No nulo, no vacío, no solo espacios
`noNulo(valor, mensaje)`	No nulo
`sonDigitos(texto)`	Todos los caracteres son dígitos

Reglas específicas:

- **DNI**: exactamente 7 u 8 caracteres, todos dígitos (**Character.isDigit()**).
- **Matrícula**: empieza con **"MP-"** seguido de 4 a 6 dígitos.
- **setEstado(null)**: lanza **IllegalArgumentException**; el estado anterior no cambia.
- **setObservaciones(null)**: no lanza excepción; las observaciones quedan como **""**.
- **Diagnóstico/tratamiento/alergia nulo o vacío**: no se registra, sin excepción.

6. Relaciones y composición

Relaciones bidireccionales

Se implementaron tres relaciones bidireccionales desde ambas puntas

Relación	Método lado "uno"	Método lado "muchos"
Hospital — Departamento	`Hospital.agregarDepartamento()`	`Departamento.setHospital()`
Hospital — Paciente	`Hospital.agregarPaciente()`	`Paciente.setHospital()`
Departamento — Medico	`Departamento.agregarMedico()`	`Medico.setDepartamento()`

Invariante: si **a** está en la lista de **b**, entonces **a.getB()** devuelve **b**.
Sin recursión infinita: **setHospital()** (y equivalentes) guarda el objeto anterior, actualiza la referencia y luego llama a **quitarX()** del anterior y **agregarX()** del nuevo. **quitarX()** es un método de visibilidad de paquete que solo hace **lista.remove()**, sin llamar de vuelta a **setHospital()**. **agregarX()** tiene un guard con **contains()** que corta el ciclo cuando el segundo extremo llama de regreso.
Sin duplicados: **agregarX()** verifica con **contains()** antes de agregar.

Composición

Dueño	Parte	Cómo se crea
`Paciente`	`HistoriaClinica`	En el constructor de `Paciente`
`Medico`	`Matricula`	En el constructor de `Medico`
`Departamento`	`Sala`	Con `Departamento.crearSala(numero, tipo)`

Los constructores de **HistoriaClinica** y **Sala** tienen visibilidad de paquete (sin modificador), lo que impide instanciarlos desde fuera de **org.jcr.entidades**.

Listas protegidas

Ningún getter de lista (**Hospital**, **Departamento**, **Sala**, **Paciente**, **HistoriaClinica**, **CitaManager**) devuelve la lista interna. Se usa **Collections.unmodifiableList(lista)**, que lanza **UnsupportedOperationException** si se intenta modificar desde afuera.

7. Programación de citas

Contrato del servicio

CitaService es la interfaz que define el contrato. **CitaManager** la implementa y guarda las citas en un **ArrayList** privado.

```
public interface CitaService {
    Cita programarCita(Paciente, Medico, Sala, LocalDateTime, double) throws CitaException;
    List<Cita> getCitasPorPaciente(Paciente paciente);
    List<Cita> getCitasPorMedico(Medico medico);
    List<Cita> getCitasPorSala(Sala sala);
}
```

CitaException

CitaException extiende **Exception** (excepción verificada, no **RuntimeException**). Esto obliga a quien llame a **programarCita()** a capturarla con **try-catch** o declararla con **throws**. Tiene dos constructores: **CitaException(String)** y **CitaException(String, Throwable)**.

Validaciones de programarCita()

programarCita() lanza **CitaException** y no registra nada si:

- El paciente, el médico o la sala son nulos.
- La fecha es nula o ya pasó (**isBefore(LocalDateTime.now())**).
- El costo es 0 o negativo.
- La especialidad del médico no coincide con la del departamento de la sala.
- El médico ya tiene otra cita a menos de 2 horas.
- La sala ya tiene otra cita a menos de 2 horas.
- Si todas las validaciones pasan, crea la cita y la registra en cuatro lugares: el ArrayList interno de CitaManager, la lista del paciente, la del médico y la de la sala.

Si todas las validaciones pasan, crea la cita y la registra en cuatro lugares: el **ArrayList** interno de **CitaManager**, la lista del paciente, la del médico y la de la sala.

Regla de las 2 horas

La ventana de exclusión se aplica en ambas direcciones. Si hay una cita a las 10:00:

Nueva cita	Diferencia	Resultado
08:30 u 11:30	90 min	Se rechaza
08:00 o 12:00	120 min	Se acepta
10:00 del día siguiente	> 24 h	Se acepta

La condición de rechazo usa **ChronoUnit.MINUTES.between()** con **Math.abs()** y la constante **HORAS_ENTRE_CITAS = 2** definida en **CitaManager**:

```
boolean seSuperponen(LocalDateTime existente, LocalDateTime nueva) {
    long minutos = Math.abs(ChronoUnit.MINUTES.between(existente, nueva));
    return minutos < HORAS_ENTRE_CITAS * 60L; // < 120: rechaza; == 120: acepta
}
```

8. Restricciones técnicas cumplidas

- Java 17, solo biblioteca estándar.
- Colecciones: únicamente **ArrayList** y **List** (**java.util**).
- Fechas: únicamente **LocalDate** y **LocalDateTime** (**java.time**).
- Sin archivos, bases de datos, Lombok, JPA, Hibernate.
- Sin **System.out.println** de depuración dentro de las entidades o servicios.
- Validaciones comunes centralizadas en **Validaciones**, sin repetición.
- Sin código muerto, sin imports sobrantes, sin variables sin usar.

9. **Resultado de MainPruebas**

Todos los **test** pasaron de forma correcta, se opta por no mostrar captura ya que la misma seria demasiado grande para la hoja, aun así, en el README.md del proyecto queda toda la documentación detallada.

Bibliografía

- Deitel, P. J., y Deitel, H. M. (2016). *Cómo programar en Java* (10.ª ed.). Pearson Educación.
- Oracle. (s. f.). Class LocalDate. En *Java SE 17 & JDK 17 documentation*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/time/LocalDate.html
- Oracle. (s. f.). Class LocalDateTime. En *Java SE 17 & JDK 17 documentation*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/time/LocalDateTime.html
- Oracle. (s. f.). Lesson: Exceptions. En *The Java Tutorials*. https://docs.oracle.com/javase/tutorial/essential/exceptions/